JavaScript 1.12

JavaScript Implementation Practice

Print-first interview refresher | contract → implementation → checks

Ask before coding
Which inputs are valid?
Which edge case separates the designs?

1 | Describe what the caller can rely on

Inputs	Outcome	Effects
types and empty cases	value, error and order	mutation and ownership

Use a small example to agree on the contract before implementing compatibility behavior.

2 | Array methods and bound calls

map	Keep length and positions. Visit an index only if it exists.
filter	Keep accepted values in order. Read each value before the predicate.
reduce	An omitted initial argument uses the first existing value.
bind	Keep the receiver for normal calls. Put saved arguments first.

Snapshot array length once. Native bind can also delegate construction to a constructable target.

3 | Promise combinators attach reactions during iteration

Method	Settlement rule	Empty input
all	All values in input order or the first observed rejection	Fulfills with an empty array
allSettled	Every outcome record in input order	Fulfills with an empty array
race	First observed settlement	Stays pending
any	First success or ordered reasons in AggregateError	Rejects with AggregateError

Reserve positions for ordered results. Normalize each element with Promise.resolve.
An iterator error rejects the combined promise. Earlier reactions are already attached.

4 | Keep one record per subscription or cache key

Event emitter

Snapshot the order and check membership.
Remove once before calling user code.

Memoization

Use has to recognize cached undefined.
Choose keys from all inputs that affect the result.

Each subscription has its own disposer. Each cache needs a policy for clearing entries.

5 | Choose how each traversal treats links

Flatten	Track arrays on the current path. Revisit shared arrays on a new path.
Group	Keep keys in a Map and append in input order.
Get / set	Validate all path segments and read own data descriptors.
Clone	Save the new object before visiting its properties. Reuse it on repeat visits.

This graph clone preserves shared references and cycles. Agree on supported types and properties.
Count unique nodes and own properties for cloning. Count each visited occurrence when flattening.

6 | Pick checks that distinguish behavior

Presence	Order	Identity
hole or undefined	finish the second promise first	shared copy and back reference

Also check iterator failure, unsubscribe during delivery and writes through invalid paths.

Explain the input size, the traversal work and the memory required by the result.